Listas, diccionarios y generadores

Una lista es una secuencia ordenada y mutable. Puede almacenar varios elementos, admitir duplicados y recorrerse con un bucle for. Una tupla también representa una secuencia ordenada, pero no permite sustituir sus elementos después de crearla. La elección depende del significado de los datos, no solo de la sintaxis.

Un diccionario asocia claves únicas con valores. Las claves deben ser hashables. Es útil para localizar un registro por un identificador o contar apariciones. Por ejemplo, conteos['SQL'] = 3 asocia el texto SQL con el número tres. Acceder a una clave ausente mediante corchetes produce KeyError; get permite especificar un valor por defecto.

Un generador produce valores a medida que se solicitan. Una función con yield devuelve un generador; no construye necesariamente toda la secuencia de antemano. Esto permite procesar un archivo grande línea por línea sin guardar todas las líneas simultáneamente en una lista. Si el código posterior convierte el generador en list, se pierde esa ventaja de memoria.

Un generador es un iterador que se consume. Tras agotarlo, volver a recorrer el mismo objeto no reinicia los datos. Para otra pasada hay que crear un nuevo generador o conservar explícitamente los resultados. La evaluación diferida también implica que ciertos errores aparecen durante el recorrido y no al construir el generador.

Errores y gestión de recursos

Las excepciones comunican que una operación no puede completarse normalmente. Un bloque try ejecuta una operación y except maneja excepciones concretas. Conviene capturar los errores que el programa sabe tratar y permitir que los inesperados se hagan visibles, en lugar de ocultar todos los fallos con except Exception: pass.

ValueError suele indicar que el tipo de un argumento es aceptable pero su valor no. Por ejemplo, int('abc') produce ValueError. KeyError indica una clave ausente en un mapeo. Una aplicación puede traducir un error técnico en un mensaje comprensible conservando la causa mediante raise NuevoError(...) from original.

El bloque with utiliza un gestor de contexto para adquirir y liberar recursos. Al abrir un archivo con with open(...), el archivo se cierra al salir del bloque, incluso si aparece una excepción. Esto facilita liberar descriptores de archivos y otras conexiones de manera predecible.

Para entradas externas hay que validar límites y formato antes de procesarlas: tamaño del archivo, extensión admitida y estructura del contenido. La extensión por sí sola no demuestra que un archivo sea un PDF válido. Los mensajes visibles para el usuario no deben incluir contraseñas ni cadenas de conexión de la base de datos.

Pruebas reproducibles y datos

Una prueba unitaria comprueba una parte pequeña de la lógica de forma aislada. Una prueba de integración comprueba la colaboración entre componentes reales, por ejemplo Python y PostgreSQL. Sustituir PostgreSQL por un objeto simulado permite probar lógica de aplicación, pero no verifica restricciones, consultas ni transacciones reales de PostgreSQL.

En pytest, una fixture prepara datos o recursos que necesitan las pruebas. Una prueba parametrizada ejecuta el mismo comportamiento con entradas diferentes. Es útil probar casos normales y límites: archivo vacío, texto ausente, carga repetida o servicio no disponible.

Un doble de prueba puede sustituir una API externa para provocar un timeout o devolver una respuesta inválida de forma controlada. Esto hace reproducibles los errores. Una prueba con dobles no demuestra la velocidad ni la calidad de un modelo real; esos aspectos necesitan medidas con el servicio y el equipo que se van a utilizar.

Separar datos de desarrollo y de evaluación final ayuda a detectar ajustes que solo funcionan en ejemplos conocidos. En un buscador documental pueden anotarse preguntas y páginas relevantes. Recall@k mide qué proporción de las fuentes relevantes aparece entre los primeros k resultados; MRR promedia el inverso de la posición de la primera fuente relevante. Estas métricas evalúan recuperación y no demuestran por sí solas que una respuesta generada sea correcta.